

Mykyta Kalchevsky

I build full-stack websites and web apps, front end to back end, and the AI automation behind them, that real businesses *run on*. My flagship is a live, bilingual Ukrainian e-commerce store; I also shipped a 7-language firmware marketplace and production Telegram bots powered by the Claude API. Six shipped projects, zero co-founders.

EMAIL: nikita.kalchevsky@gmail.com | TELEGRAM: [@Hybrid_d](https://t.me/Hybrid_d) | PHONE: +1 (239) 887-9992 | GITHUB: [@nikitakalchevsky](https://github.com/nikitakalchevsky) | LINKEDIN: [nikitakalchevsky](https://www.linkedin.com/in/nikitakalchevsky)

About

// who · what · why

15 years building and running networks taught me one thing: *code is easy, keeping it alive is hard*. Now I bring that operations mindset to full-stack web development, building websites, web apps, and the automation behind them, front end to back end. I work with React, Next.js, Firebase, and Python, ship to Vercel and DigitalOcean, and treat every project like it has paying customers behind it. Because mine do.

Selected Work

// 2024 - 2026

FLAGSHIP · LIVE · PRODUCTION

Apelsin Rozmarin, Full-Stack E-Commerce

My flagship build: a full-stack e-commerce store for the Ukrainian market, designed, coded, deployed, and maintained end to end, solo. React storefront on the front, Firebase on the back: 441 live products, a custom admin panel, a real-time order pipeline, and three payment methods. Indexed in Google Merchant Center, SSL grade A.



441 products · 2 languages · 3 payment rails · SSL grade A · Solo build

stack: React · Vite · Firebase (Firestore, Auth, Hosting) · Telegram Bot API · Monobank · PayPal

- ➔ **Front to back, solo:** React storefront, Firebase data and auth, custom admin panel for products, inventory, and orders
- ➔ **Bilingual storefront:** full UI localized in two languages for the Ukrainian market
- ➔ **Real-time Telegram notifications** dispatched to operator chats on every order
- ➔ **Three payment rails:** Monobank acquiring, IBAN bank transfer, PayPal
- ➔ **SEO & commerce:** Google Search Console plus Merchant Center, sitemap submitted, SSL A grade

LIVE · PRODUCTION

TuningStore, ECU Firmware Marketplace

A multilingual digital marketplace for ECU calibration files: stock firmware, Stage 1/2 tunes, DPF/EGR/AdBlue off, gearbox maps. Customers browse a deep hierarchical catalog (brand, model, engine, calibration), purchase securely, and receive a presigned R2 download link. Built solo from zero.



7 languages · 301+ files · 3 payment rails · R2 secure delivery

stack: Next.js 16 (App Router) · TypeScript · Prisma 7 · PostgreSQL (Neon) · NextAuth v4 · Stripe · PayPal · Coinbase · Cloudflare R2 · next-intl (7 locales)

- ➔ **4-level catalog:** brand, model, engine, calibration file, 301+ firmware files imported
- ➔ **7 languages:** EN, UA, RU, DE, FR, ES, IT via next-intl with full route localization
- ➔ **Three payment rails:** Stripe, PayPal, Coinbase Commerce, webhooks handle order fulfillment
- ➔ **Secure delivery:** Cloudflare R2 presigned URLs, files never exposed publicly
- ➔ **Admin bulk import:** drag-and-drop .bin upload, auto-parse filename, DB plus R2 in one shot

LIVE · COMMERCIAL

Airbag ECU Lookup Bot

A production Telegram bot serving real customers for an automotive ECU programming service in Glendale, CA. Customers send a vehicle brand and part number; the bot queries a 16,003-record database, returns the correct reprogramming method and price, and books an appointment within an hour. Claude Haiku handles messy natural-language inputs.



stack: Python · Claude Haiku API · python-telegram-bot · openpyxl · DigitalOcean · Linux/cron

- ➔ **Two-bot architecture:** client-facing bot plus admin management panel
- ➔ **Production deploy:** Ubuntu droplet, persistent screen sessions, cron auto-restart every 6h
- ➔ **Lean ops:** 89MB memory footprint, runs comfortably on 1GB RAM / 1 vCPU
- ➔ **API tuning:** eliminated Claude 400 errors via prompt plus DB chunking and temperature 0.1

R&D · MODULAR

Personal AI Assistant

A modular Telegram-based personal assistant where Claude Haiku handles all natural-language input parsing. "Ate 200g chicken" becomes a structured calorie entry; "spent \$40 on gas" becomes a categorized expense. Built as a testbed for production patterns I now reuse for client work.



stack: Python · Claude Haiku · SQLite · python-telegram-bot · cron

- ➔ **Nutrition module** with Mifflin-St Jeor BMR calculation and personalized macro goals
- ➔ **Finance tracking** with category breakdowns and monthly statistics
- ➔ **Schedule & reminders,** timezone-aware
- ➔ **Copart auction monitor** polling every 2 hours for new listings
- ➔ **Reliability:** weekly SQLite backups, auto-restart on reboot via cronab

LIVE · TRADING

Smart DCA Trade Bot

Automated crypto DCA strategy bot for Binance.US, managed entirely via Telegram inline menus, no web interface needed. Scans top coins by volume and momentum, buys on dips below MA20, scales position size with dip depth, and auto-sells on take-profit. Deployed on DigitalOcean with non-blocking async exchange calls.



stack: Python · ccxt (Binance.US) · python-telegram-bot · APScheduler · SQLite · asyncio · DigitalOcean

- ➔ **Smart DCA strategy:** buys when price < MA20 by a configurable threshold, scales position size with dip depth
- ➔ **Auto-mode:** autonomous buy/sell within limits (max invested, min interval between trades)
- ➔ **Auto-sell only mode:** takes profit on open positions without enabling auto-buy
- ➔ **Inline Telegram UI:** positions, P&L, settings, full control without a web dashboard
- ➔ **DRY_RUN mode** for safe strategy testing without real orders

FRAMEWORK · INFRASTRUCTURE

LangChain Framework + Bot Fleet

A LangChain-powered Telegram assistant designed as a deployment-ready foundation for client-specific automation. A modular skills directory keeps each capability isolated and swappable per client. It currently runs alongside three other production bots on a single S5 droplet: 11 days uptime, zero incidents, load average 0.00.



stack: Python · LangChain · ChatAnthropic · screen · cron · modular skills architecture

- ➔ **Skill-based modular design:** database, messaging, logging, approval, pricing, orders, inventory
- ➔ **Approval workflows** for human-in-the-loop scenarios
- ➔ **Fleet management:** 4 production bots co-located on one droplet, isolated screen sessions
- ➔ **Reliability proven:** production uptime measured in days, not hours

Stack

// daily drivers

FRONTEND	BACKEND	AI & LLM	INFRASTRUCTURE
----------	---------	----------	----------------

React · TypeScript	Node.js · Next.js API	Claude API (Sonnet, Haiku, Opus)	DigitalOcean droplets
--------------------	-----------------------	----------------------------------	-----------------------

Next.js (App Router)	Python 3	Anthropic SDK	Vercel · Cloudflare R2
----------------------	----------	---------------	------------------------

Vite	Prisma ORM · PostgreSQL	LangChain	Linux / Ubuntu
------	-------------------------	-----------	----------------

JavaScript (ES6+)	Firebase (server, Auth)	Prompt engineering	Stripe · PayPal · Coinbase
-------------------	-------------------------	--------------------	----------------------------

HTML5 / CSS3	python-telegram-bot	Function calling / tools	screen · cron · SSH
--------------	---------------------	--------------------------	---------------------

next-intl · i18n	APScheduler · SQLite	Claude Skills	Git / GitHub
------------------	----------------------	---------------	--------------

Firebase (Firestore, Auth)	async / await	Workflow automation (n8n, Make, Zapier)	DNS · Search Console
----------------------------	---------------	---	----------------------

WordPress			
-----------	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--

--	--	--	--